

LearnUp

A multi-tenant, retrieval-grounded Q&A portal for institutions

Project	LearnUp — Prototype
Type	Web-based multi-tenant SaaS portal
Domain	EdTech · Retrieval-Augmented Generation (RAG)
Stack	HTML5, CSS3, Vanilla JavaScript (ES6), Inline SVG
Author	Dharnis M — B.Tech CSE, 3rd Year
Document	Project Structure & Architecture Reference

1. Problem & Motivation

College students constantly ask repeated questions about their own syllabus, previous-year question papers (PYQs), notes and course outcomes. The answers exist in PDFs and slide decks already uploaded by faculty — but there is no queryable interface over them. Generic LLMs like ChatGPT can talk about the topic in general, but cannot cite the specific document a lecturer uploaded, and cannot be locked to one institution's private material.

Goal

Build a portal where each institution has its own private knowledge base (KB) of uploaded documents, and where students can chat with an assistant that answers **only** from that institution's material — with source citations on every answer, and hard tenant isolation between institutions.

2. Project Structure

Directory layout of the prototype (rooted at D:\learnup\):

```
learnup/
|
|-- index.html           Tenant landing page: chooses Student vs Faculty portal
|-- student.html         Student experience: pick institution -> chat with its KB
|-- organization.html    Admin dashboard (Overview / Resources / Student activity)
|-- styles.css           Shared design tokens, layout, components
|-- data.js             Demo orgs + documents + fake RAG matcher (matchKB)
|
|-- README.md           Project overview and how-to-run
|-- DEMO_SCRIPT.md       4-minute presentation walkthrough
+-- LearnUp_Architecture.pdf This document
```

File responsibilities

File	Responsibility
index.html	Landing / portal chooser. Branded per tenant.
student.html	Institution picker + chat UI + streaming-style typing indicator + citation chips + isolation-denial rendering.
organization.html	Admin sidebar layout with three tabs: Overview (stats + query trends chart), Resources (upload / search / delete), Student activity (query log).
styles.css	Design tokens (CSS custom properties), buttons, cards, tables, badges, chat bubbles, chart styling, responsive breakpoints.
data.js	ORGS registry (two demo tenants), each org's docs[] / kb[] / otherOrgKeywords[]. Exports matchKB(orgId, q).

3. High-Level Architecture

LearnUp follows a three-layer architecture: a Presentation layer (the two portals), an Application layer (routing, chat orchestration, upload pipeline), and a Retrieval layer (per-tenant vector search with hard isolation).

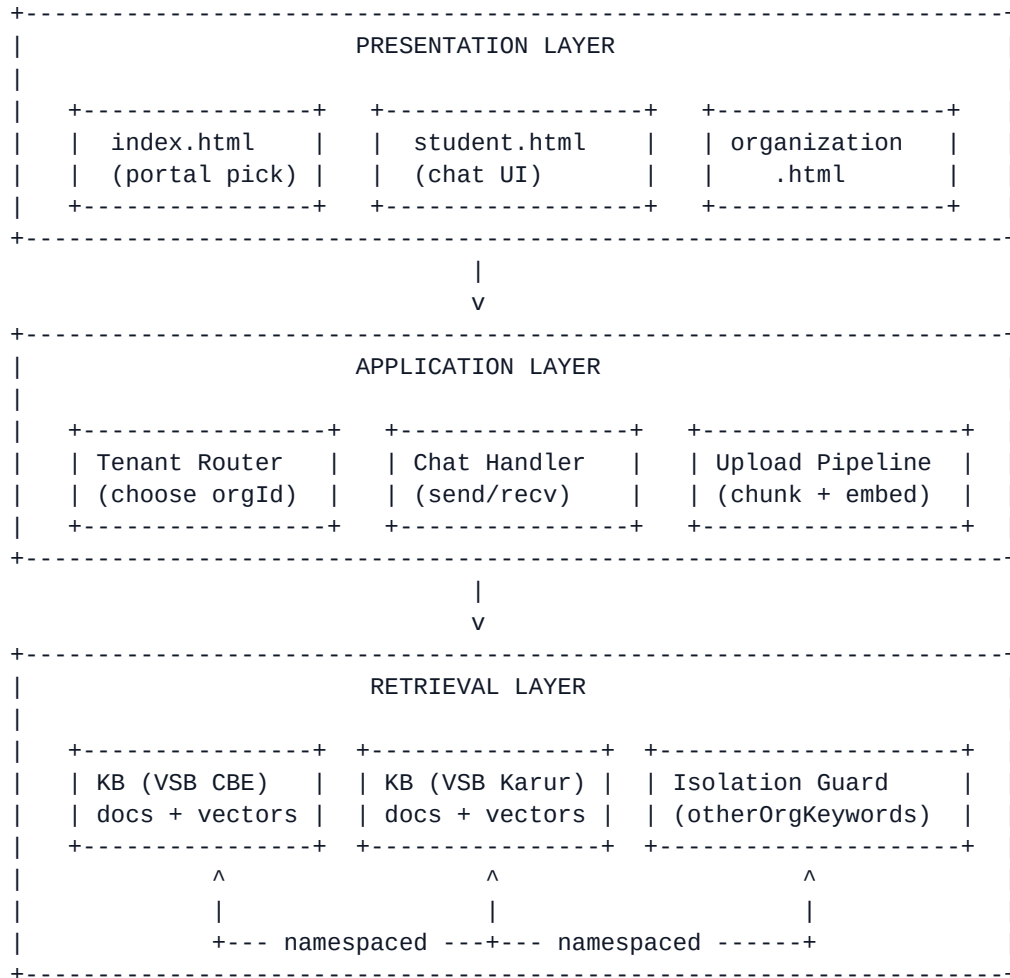


Fig. 1 — Three-layer architecture with per-tenant retrieval namespaces.

4. Component Architecture

4.1 Student Portal

The student portal is a two-screen SPA-style flow. First the institution picker; then a chat interface with suggested questions, typed input, citation chips beneath every answer, and a red-tinted bubble for denied cross-tenant queries.

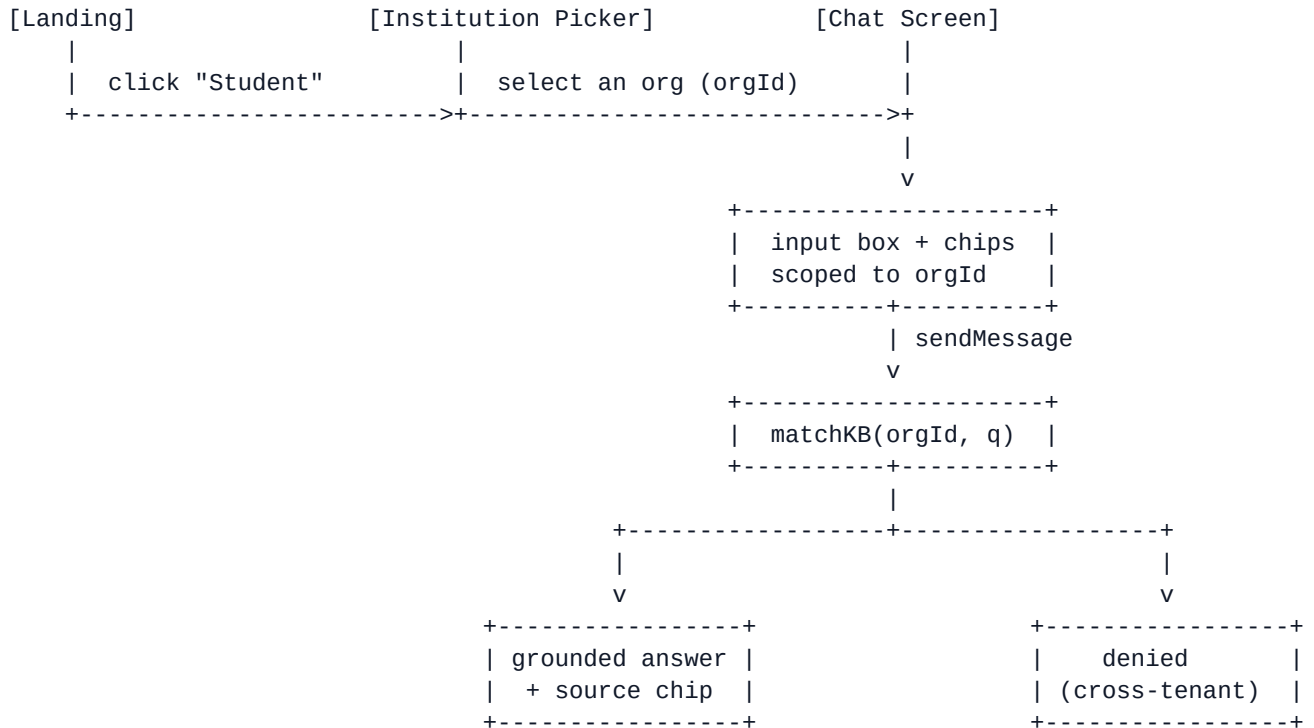


Fig. 2 — Student portal navigation and message flow.

4.2 Admin Portal

The admin portal is a classic sidebar-plus-main layout. Three tabs share the same shell; each renders a distinct concern.

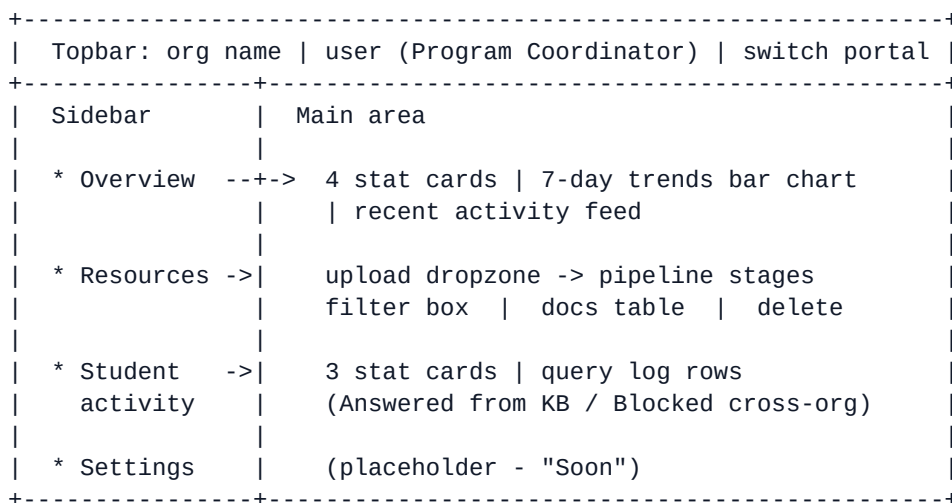


Fig. 3 — Admin portal tab layout.

5. Data Model

All demo state lives in `data.js`. The shape below is what a production Postgres schema would eventually mirror.

```
ORGS = {
  <orgId>: {
    id:      string,      // "vsb", "nexus"
    name:    string,      // display name
    short:   string,      // 2-3 char label for avatar
    type:    string,      // "Engineering College, Autonomous"

    docs: [                // uploaded documents (metadata only in demo)
      { name, ext, size, chunks, uploaded }
    ],

    suggested: [ string ], // chips shown above the chat input

    kb: [                  // FAKE retrieval index (keyword -> answer)
      {
        kw: [ string ],    // trigger keywords
        a:  string,        // grounded answer text
        src: [{ d, p }]    // citation: document name, page/section
      }
    ],

    otherOrgKeywords: [    // topics that BELONG to another tenant
      string               //   -> matchKB returns { denied: true }
    ]
  }
}
```

Production mapping

Prototype (data.js)	Production equivalent
ORGS[orgId]	tenants table, row per institution
docs[]	documents table (org_id FK, storage_uri, status)
kb[] with kw/a/src	chunks table + vector index (pgvector / Qdrant) filtered by org_id
otherOrgKeywords	Not needed — namespace-scoped ANN search physically cannot cross tenants
matchKB() keyword scan	embed(query) -> top-k ANN search -> LLM with retrieved context + system prompt

6. Key Flows

6.1 Chat query — answered from KB

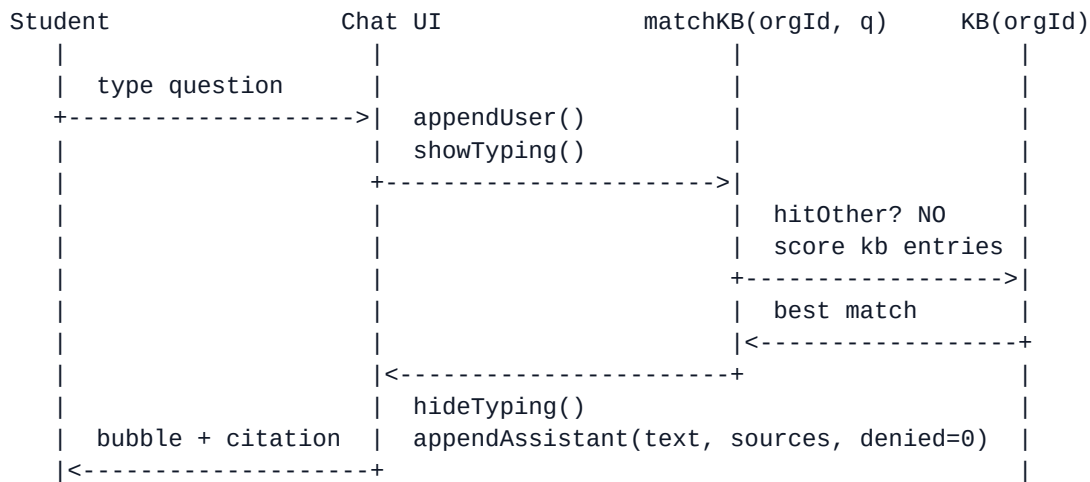
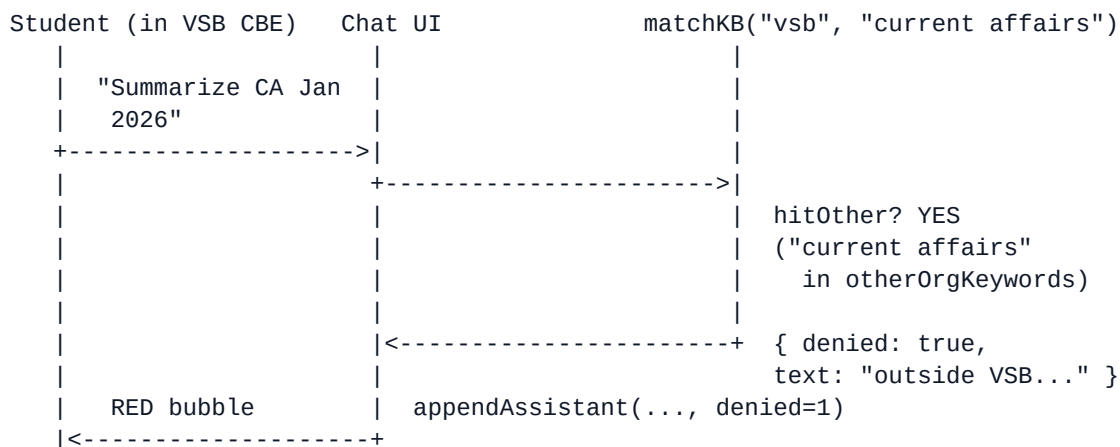


Fig. 4 — Successful retrieval and citation.

6.2 Chat query — cross-institution DENIED (isolation guarantee)



The KB is NOT consulted. In production, the retrieval namespace filter enforces this at the vector-store layer, not the app layer.

Fig. 5 — Cross-tenant refusal — the core product guarantee.

6.3 Document upload pipeline

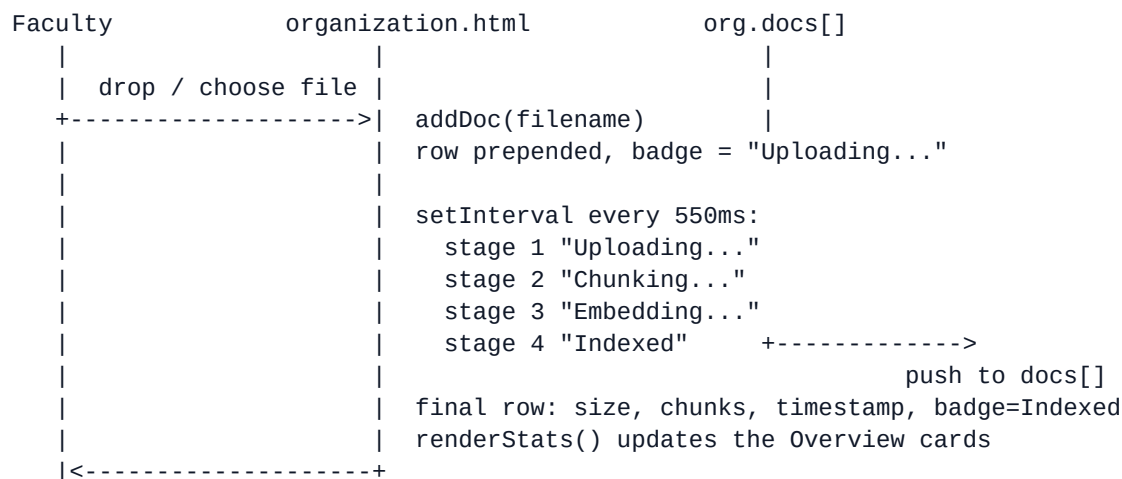


Fig. 6 — Simulated ingestion pipeline mimicking real chunk/embed/index stages.

7. Tech Stack

Layer	Prototype	Production target
UI	HTML5 + CSS3 + Vanilla JS	React (Vite) + TypeScript
Styling	CSS custom properties	Tailwind CSS or Styled Components
Icons / charts	Inline SVG (no library)	Lucide icons + Recharts
Backend API	(none — static)	FastAPI (Python) or Node/Express
Auth	(mocked user card)	Clerk / Auth0 + org-scoped roles
Relational DB	(none)	PostgreSQL (tenants, docs, users)
Vector store	kb[] array + keyword scan	pgvector or Qdrant, namespaced
Embeddings	(faked)	OpenAI text-embedding-3-small
LLM	hard-coded answer strings	Claude / GPT-4-class w/ RAG prompt
Object storage	(none — filename only)	AWS S3 (per-tenant prefix)
Deployment	open index.html locally	Vercel (UI) + Fly.io / Render (API)

8. Non-Functional Requirements

NFR	How the prototype addresses it
Tenant isolation	Every retrieval carries orgId. Cross-tenant topics trigger an explicit denial. In production, the vector-store namespace filter enforces this at the storage layer.
Groundedness	Every assistant answer renders a source chip (document + page/section). Unanswerable queries return a helpful list of indexed topics instead of hallucinating.
Observability	Admin 'Student activity' tab logs every question with Answered / Blocked status, so faculty can see what students actually ask.
Performance	Static assets, no build step, sub-second response with simulated latency. Production would target p95 < 2s for the full RAG round-trip.
Responsiveness	Media query at 860 px collapses the sidebar and reflows stat grid to 2 columns. Chat width capped for readability.
Accessibility	Semantic HTML, proper labels on inputs, sufficient colour contrast on all badges.

9. Future Work

- 1. Real ingestion pipeline.** Replace the simulated stages with actual PDF/DOCX parsing (unstructured.io), overlapping 500-token chunking, and embedding via OpenAI or a local model.
- 2. Real retrieval.** Swap the keyword matchKB() for pgvector-backed ANN with cosine similarity, top-k = 5 with a re-ranker.
- 3. Authentication & RBAC.** Wire in Clerk / Auth0 with an org-scoped role model (Student, Faculty, Coordinator, Admin).
- 4. Streaming responses.** Replace the typing dots with true SSE token streaming from the LLM.
- 5. Feedback loop.** Thumbs up/down on answers, feeding into a review queue faculty can use to correct or extend the KB.
- 6. Analytics.** Real time-series charts, top-queried documents, unanswered-question digest.

10. Conclusion

LearnUp demonstrates that the value of RAG for education is not just the answer quality — it is the **tenant boundary**. A student in one college getting an answer sourced from another college's private

material would be a product failure, no matter how correct the answer. This prototype makes that boundary visible and testable: the same question changes from an answer to a refusal purely by switching institutions. Everything else — the upload pipeline, the citation chips, the query log — is the surrounding UX that makes the boundary usable.